

Spring Boot

Spring Boot & Microservices

- 1) Your trainer
- 2) Pre-requisites
- 3) Who should join
- 4) Course Content (Road map)
- 5) Course Details
- 6) Q & A

Trainer

Name: Mr. Ashok

11+ Yrs exp in IT

Product Based in Hyd (manager)

8+ Yrs exp Training

skills : Java frameworks

Linux

DevOps

AWS cloud

started Ashok IT - 2020

Pre-requisites

- 1) Core Java (1.8v)
 - oops

- collections
- exception handling
- io streams
- multi threading
- java 8 features

2) Adv Java

- JDBC
- Servlets
- JSP (not required)
- MVC
- Web Server (Tomcat)

3) Database

- SQL

4) Web Technologies

- HTML & CSS

Who should join for this course ?

- ❖ Every java developer should know sbms
 - freshers
 - career gap students
 - java working professionals

Course Content

Module-1 : Spring Core

Module-2 : Spring Boot

Module-3 : Data JPA (extension for hibernate)

Module-4 : Spring Web MVC (C2B)

Module-5 : Spring REST (webservices)

Module-6 : Microservices

Module-7 : Spring Cloud

Module-8 : Spring Security

Module-9 : Realtime Tools (weekends)

- junit

- logging

- kafka

- redis

- docker

- jenkins

- swagger

- postman

=====

Course : Spring Boot with Microservices

Course Code : 45-SBMS

Class Timings : 9:30 AM - 11:00 AM (IST)

(Mon-Mon)

Course Duration : 3 Months

Soft copy notes on daily basis

Course Fee : 8000 INR (live classes + notes)

Notes: For backup videos : 2000 INR (1 year validity)

1) Resume Building

2) Topic wise FAQ's

3) Mock Interviews

Core Java vs Adv Java vs Java Framework

Core Java

- ❖ Core java means J2SE
- ❖ Core Java is base to become java developer
- ❖ In core java fundamentals will be available
 - data types
 - arrays
 - strings
 - oops
 - collections
 - exception handling
 - io streams
 - lambdas
 - stream api

- threads

- ❖ using core java we can develop standalone apps (one user can access a time)

Ex : notepad, calc, eclipse, intellij idea....

Adv Java (JEE)

- ❖ JEE developed based on JSE only

- ❖ JEE is called as Java Technology

- Servlets

- JSP

- ❖ Using this JEE we can develop web applications (multiple users can access at a time thru internet)

Ex: gmail, facebook, naukri, youtube

- ❖ To run web application server is required

Ex: Tomcat, glassfish, jboss, jetty, netty

1) Web Servers (Ex : Tomcat)

2) App Servers (ex: JBOSS, Web Logic, Web Sphere)

- ❖ Note: App Server provides more functionalities than web server

Ex : EJB container, security, conn pooling....

- ❖ Application can be developed in 2 ways

1) Monolithic (outdated)

2) Microservices (trending)

Monolithic

- ❖ Developing all the functionalities in single app
- ❖ We have below challenges in monolithic
 - 1) Maintenance Difficult
 - 2) Single point of failure
 - 3) Re-Deploy entire app
 - 4) Technology dependent
- ❖ To overcome problems of monolith, we are using microservices architecture.

Microservices

- ❖ It is an approach to develop application functionalities with loosely coupling.
- ❖ Application functionalities will be developed in multiple services/apis.
- ❖ Every microservice will run in separate server
- ❖ Every microservice is independently deployable and executable.
- ❖ We have below advantages with microservices
 - 1) Functionality distributed
 - 2) Loosely coupled
 - 3) No single point of failure
 - 4) Technology independent

Java Frameworks

- ❖ Framework is a semi developed software
- ❖ Frameworks providing common logics for application development
- ❖ If we use frameworks then we need to focus only on business logic
- ❖ Using frameworks we can do more work in less time (productivity)
- ❖ We have several frameworks in java community
 - 1) JSF -----> outdated
 - 2) Hibernate -----> data jpa
 - 3) Struts -----> out dated
 - 4) Spring ---> SpringBoot ----> trending

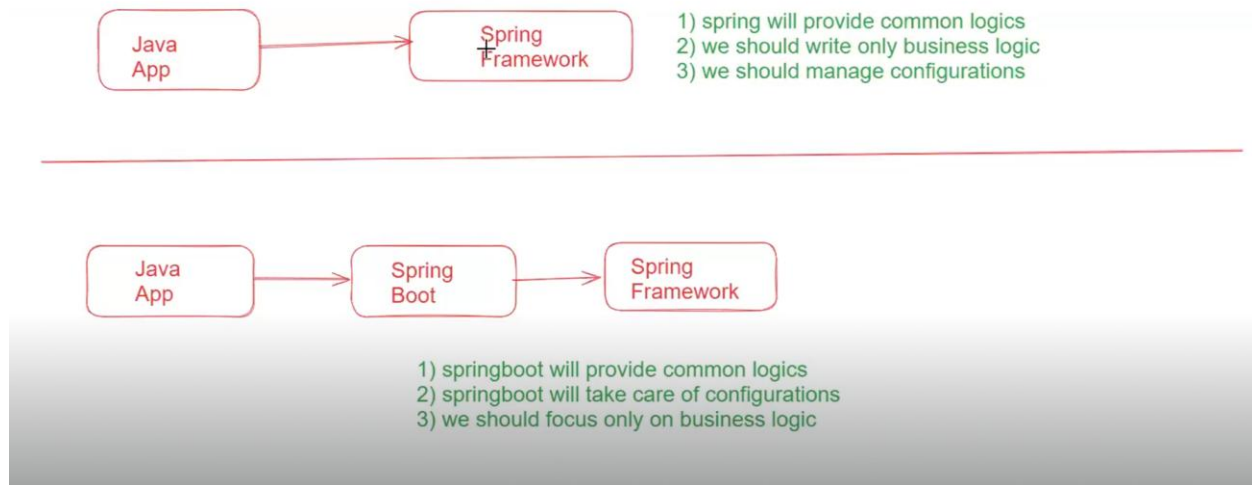
Note: The above frameworks developed by using JSE and JEE.

- ❖ Hibernate is an ORM framework.
- ❖ ORM means Object Relational Mapping
- ❖ Using Hibernate we can develop persistence layer
- ❖ Hibernate developed on top of JDBC.

- ❖ Struts framework developed by Apache
- ❖ Struts is used to develop only web layer

- ❖ Spring is called as application development framework
- ❖ Using spring we can develop entire application.
- ❖ Spring is free of cost & open source
- ❖ Spring framework provides common logics required for application development.

Note: SpringBoot is an extension for Spring Framework.



Date : 26-Jul-2024

Topic : Spring Boot Introduction

- ❖ Spring Boot is an extension for spring framework.
- ❖ Spring Boot is an approach to develop spring based application with less configuration.
- ❖ Using Spring boot we can develop several types of applications
 - a) standalone applications
 - b) web applications (C 2 B)
 - c) distributed applications (B 2 B)
- ❖ If we develop project using spring then we need to manage configurations.

- ❖ If we develop project using spring boot then we will get Auto Configuration.
- ❖ Autoconfiguration means spring boot will identify the configuration required for the project and it will provide that configuration in runtime of our application.

Note: Spring boot will identify configurations required for the project and it will provide those configurations.

- starting ioc container
- component scanning
- create connection pool (data source)
- start embedded server
- provide default security
- provides actuators to monitor our app

- ❖ Using spring boot we can achieve rapid application development.

Advantages with spring boot

1) pom starters (dependencies)

- a) web-starter
- b) jpa-starter
- c) security-starter
- d) mail-starter
- e) actuator-starter etc...

Note: pom starters are used to enable auto configurations in boot application.

2) Auto Configuration

- ❖ Based on pom starters boot will identify configurations required for the project and boot will manage those configuration.

web-starter ==> tomcat server

data-jpa-starter ==> db connection pooling

security-starter => default login page

actuator-starter => monitoring

3) Embedded Servers/Containers

- ❖ Springboot will take care of servers to run our web applications (we no need to download & install)
- ❖ Springboot will support 3 embedded containers

a) tomcat (default)

b) jetty

c) netty

4) Actuators (production read features)

- ❖ Actuators are used to monitor and manage our applications.
 - check health
 - check beans loaded
 - check url patterns
 - check threads
 - check heap memory

5) Component Scanning

- ❖ Using component scanning spring boot will scan our project packages and it will identify spring beans.

How to create spring boot project ?

- ❖ Approach-1 : Create boot application using spring initializer website then download it and extract it and import into eclipse / intelliJ IDE

URL : start.spring.io

Note: IntelliJ ultimate version will support springboot project creatin directley. IntelliJ ultimate is licensed.

- ❖ Approach-2 : Use STS ide to create springboot application directley.

Note: STS ide will communicate with intializr website to download project.

Note: To create springboot application internet is mandatory.

How to install STS IDE

1) download sts ide jar file

STS Download Link : https://cdn.spring.io/spring-tools/release/STS4/4.24.0.RELEASE/dist/e4.32/spring-tool-suite-4-4.24.0.RELEASE-e4.32.0-win32.win32.x86_64.self-extracting.jar

2) run that jar file from cmd

syntax : java -jar <file-name>

3) Go to sts folder and open SpringToolSuite4.exe file

How to install IntelliJ IDE

1) Download IntelliJ community version

Download Link : <https://www.jetbrains.com/idea/download/download-thanks.html?platform=windows&code=IIC>

2) Once .exe file is downloaded double click and install it.

Date : 27-Jul-2024

Topic : Spring Boot project

Agenda of Day

1) Creating Spring boot application

2) Playing with pom starters

3) Understanding spring boot project folder structure

4) Running spring boot application

5) Internals of Spring boot (postmortem for start class)

Spring boot project folder structure

spring-boot-app

- **src/main/java** =====> project source code
 - Application.java =====> main/start class
- **src/main/resources** =====> config files (xml,yml,props)
 - static (images goes here,css,js)
 - templates (html files goes here)
- application.properties =====> configurations
- **src/test/java** =====> unit test classes (junits)
 - ApplicationTests.java =====> Junit class
- **maven-dependencies** =====> jars downloaded
- **target** (.class files goes here)
- **pom.xml** =====> maven config file

Date : 29-Jul-2024

Topic : Spring Boot Internals

- 1) What is start class in spring boot
- 2) What is run () method in spring boot
- 3) Internals of run () method
- 4) what is @SpringBootApplication annotation
- 5) What is component scanning ?

What is start class in spring boot ?

- ❖ Start class is also called as main class of spring boot
- ❖ This start class will be created by default when boot app got created.
- ❖ It is entry point for boot application execution. Execution will start from here only.

Q) How IOC container will be started in Spring boot application ?

- ❖ `run ( )` method will take care of starting IOC container.
- ❖ `run ( )` will use below classes to start IOC container based on starter available in `pom.xml`

`spring-boot-starter :: AnnotationConfigApplicationContext`

`spring-boot-starter-web :: AnnotationConfigServletWebServerApplicationContext`

`spring-boot-starter-webflux :: AnnotationConfigReactiveWebServerApplicationContext`

Note-1: when we use web-starter boot will give "tomcat" as default embedded container.

Note-2: When we use webflux-starter boot will give "netty" as default embedded container.

What is banner in spring boot ?

- ❖ When we run boot application by default spring logo will be printed on console that is called as banner in spring boot.

Note: `run ( )` method contains logic to print the banner.

- ❖ We can change banner text according to our requirement by creating "banner.txt" file in "src/main/resources" folder.
- ❖ Spring boot banner works based on modes. We have 3 types of modes here
 - 1) console (default)
 - 2) log
 - 3) off
- ❖ We can set banner mode using "application.properties" file

```
spring.main.banner-mode=off
```

What is return type of run () method ?

- ❖ run () method returns ioc container obj using ConfigurableApplicationContext (interface)
ConfigurableApplicationContext context =

```
SpringApplication.run(Application.class, args);
```

Note: in the above code, context variable is holding interface impl class obj.

What is runner in springboot ?

- ❖ runners are used to execute the logic only one time when boot application started.

How run () method works internally ?

- start the timer
- create bootstrap context
- load listener classes
- start the listeners
- prepare environment (read props/yml file)
- create application-context (ioc)
- prepare and refresh IOC (DI)
- stop the timer
- calculate time taken and print on console
- call runners
- return IOC obj

Assignment

What is @SpringBootApplication annotation ?

What is Component Scanning ?

Date : 30-Jul-2024 Topic : Component scanning ? @Bean & @SpringBootApplication annotation

Q) How to represent java class as Spring Bean ?

- ❖ We can use below annotations to represent java class as spring bean.

@Component

@Service

@Repository

@Configuration-----> java class as configuration class

@Bean -----> method level annotation

@Controller

@RestController

Note: Spring bean classes will be managed by IOC container.

Q) What is component scanning ?

- ❖ It is the process of identifying spring beans available in the project.
- ❖ Component Scanning will start from base package (the package which contains start class).
- ❖ Once base package scanning completed then it will go for sub packages of base package.

Note: Any package name which is starting with base package is called as sub package.

Ex:

in.ashokit----- will be scanned

- Application.java

in.ashokit.dao ----- will be scanned

in.ashokit.service ----- will be scanned

in.ashokit.security ----- will be scanned

com.oracle.util ----- will not be scanned

Note: We can configure more than one base package using @ComponentScan annotation like below.

@SpringBootApplication

@ComponentScan(basePackages = {"in.ashokit","com.oracle.utils"})

```
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```

What is @Bean annotation ?

❖ It is method level annotation. It is used to customize bean obj creation.

@Configuration

```
public class AppConfig {
    @Bean
    public AppSecurity createAppSecurity() {
        System.out.println("security app method called....");
        AppSecurity security = new AppSecurity("AES");
        return security;
    }
}
```

Note: It is recommended to keep @Bean methods in configuration class. SpringBoot start class also acts as configuration class.

Q) What is @SpringBootApplication annotation ?

- ❖ This is used at start class of the springboot.
- ❖ This annotation is equal to below 3 annotations
 - @SpringBootConfiguration
 - @EnableAutoConfiguration
 - @ComponentScan

Assignment :

- 1) Bean Scopes
- 2) Bean Life Cycle

Date : 31-Jul-2024

Topic : Bean Scopes

Bean Scopes

- ❖ Bean scope will represent how many objects should be created for a spring bean.
- ❖ We have below 4 scopes
 - 1) singleton (default)
 - 2) prototype
 - 3) request
 - 4) session

- ❖ If we don't specify any scope then by default it will consider as singleton scope.

Note: For singleton scoped beans only one object will be created. Singleton beans are eager loading beans that means when ioc container started then immediately objects will be created for singleton beans.

Note: When we call `getBean ( )` method IOC will return existing obj of singleton bean.

- ❖ Prototype beans are lazy loading beans that means when we call `getBean()` method then only new object will be created. Multiple objs will be created for prototype beans.

```
@Component
```

```
@Scope("prototype")
```

```
public class UserService {
```

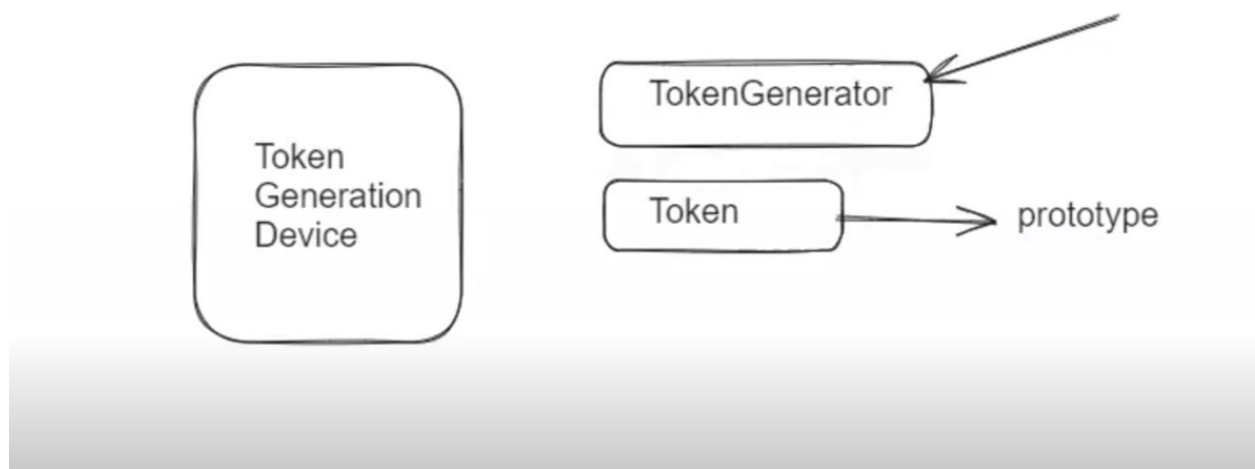
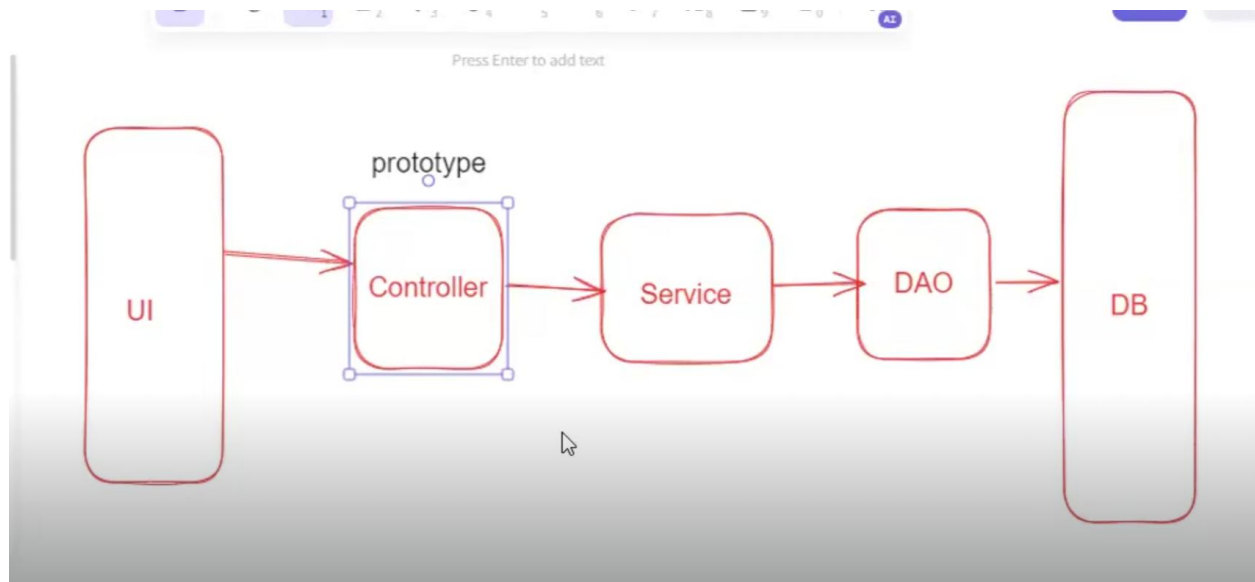
```
    public UserService() {
```

```
        System.out.println("UserService :: Constructor");
```

```
    }
```

```
}
```

Note: request and session scopes are related to spring web mvc module.



Summary

- 1) What is Spring boot
- 2) Advantages with Spring Boot
 - pom starters
 - auto configuration

- embedded servers
- actuators
- component scanning

3) Spring Boot application development

4) Spring Boot application project folder structure

5) Start class in spring boot

6) Internals of SpringApplication.run () method

7) What is @SpringBootApplication annotation

8) What is component scanning

- base package
- sub package

9) How to represent java class as Spring Bean

- @Component
- @Configuration
- @Bean

10) Bean Scopes

- singleton
- prototype
- request
- session

Assignment :

Autowiring

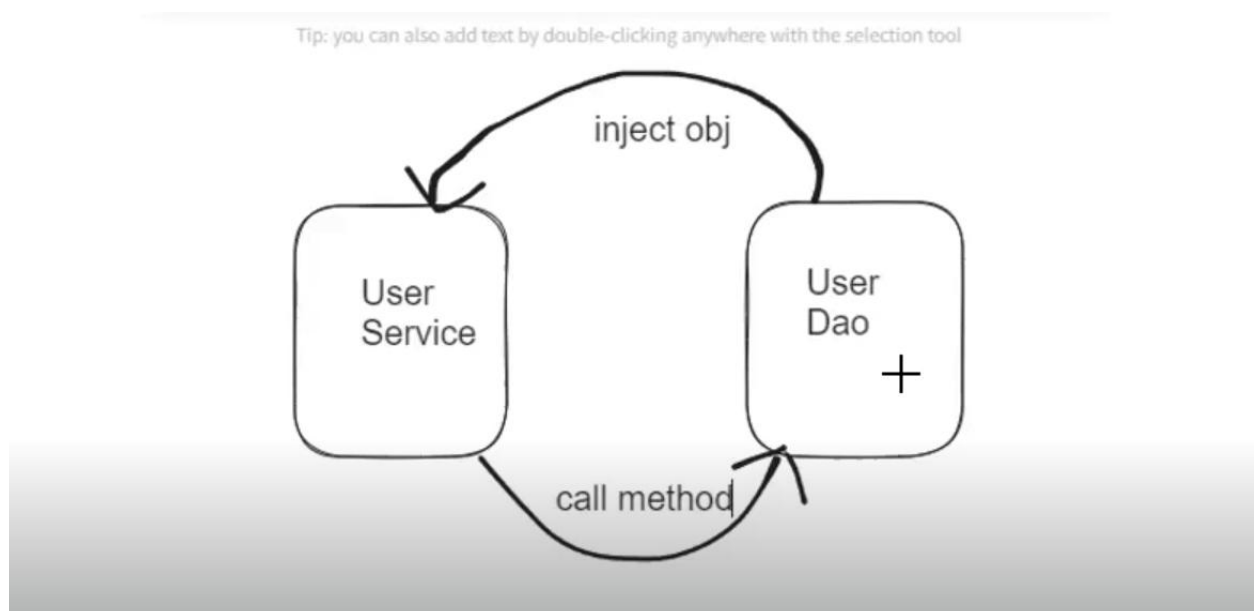
Date : 02-Aug-2024

Topic : Autowiring

- ❖ The process of injecting one class obj into another class obj is called as dependency injection (DI).

Ex:

- ◆ controller should call service method (inject service obj into controller)
- ◆ service should call dao method(inject dao obj into service)



- ❖ IOC container is responsible to perform dependency injection in our applications.
- ❖ DI we can perform in 3 ways
 - 1) Setter Injection (SI)
 - 2) Constructor Injection (CI)
 - 3) Field Injection (FI)

- ❖ Autowiring is used to perform Dependency Injection.
- ❖ TO perform DI with Autowiring we will use @Autowired annotation
- ❖ @Autowired annotation we can use at 3 places
 - setter method level (SI)
 - constructor level (CI)
 - field/variable level (FI)

@Autowired

```
private UserDao dao;
```

@Autowired

```
public void setDao(UserDao dao){  
    this.dao = dao;  
}
```

@Autowired

```
public UserService(UserDao dao){  
    this.dao = dao;  
}
```

- ❖ Autowiring works based on modes

1) byName --> based on variable name it will identify dependent bean object

```
private UserDao dao;
```

2) byType -> based on data type

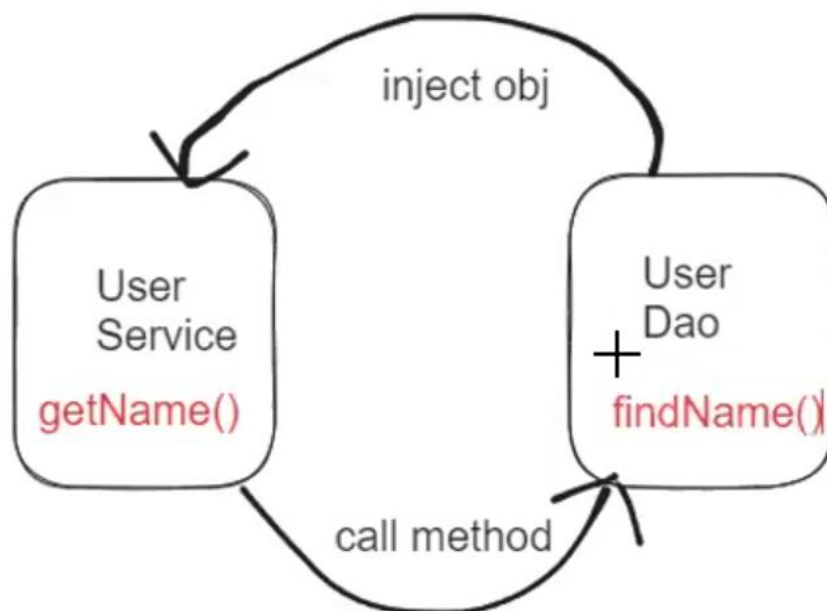
3) constructor

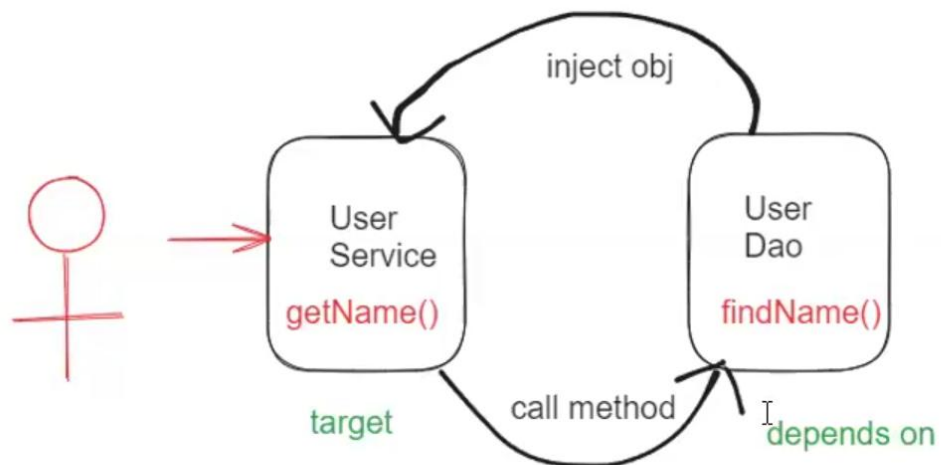
4) none

When we perform Autowiring based on byType then chance to get ambiguity, we can solve ambiguity using below 2 ways

@Qualifier(name)

@Primary -> class level



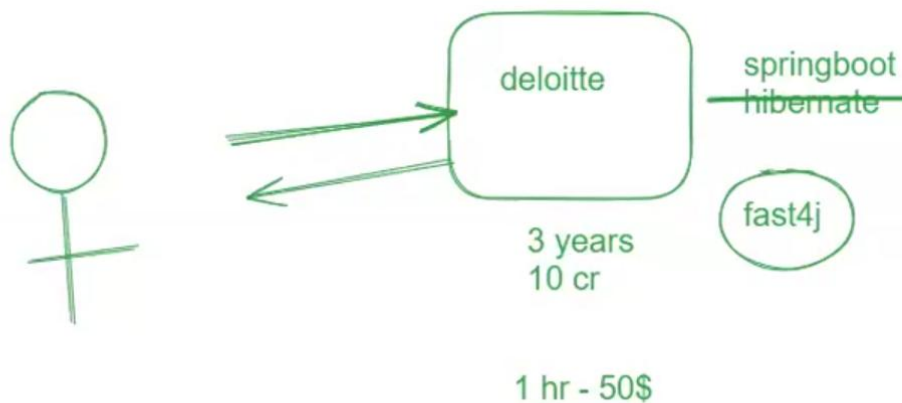


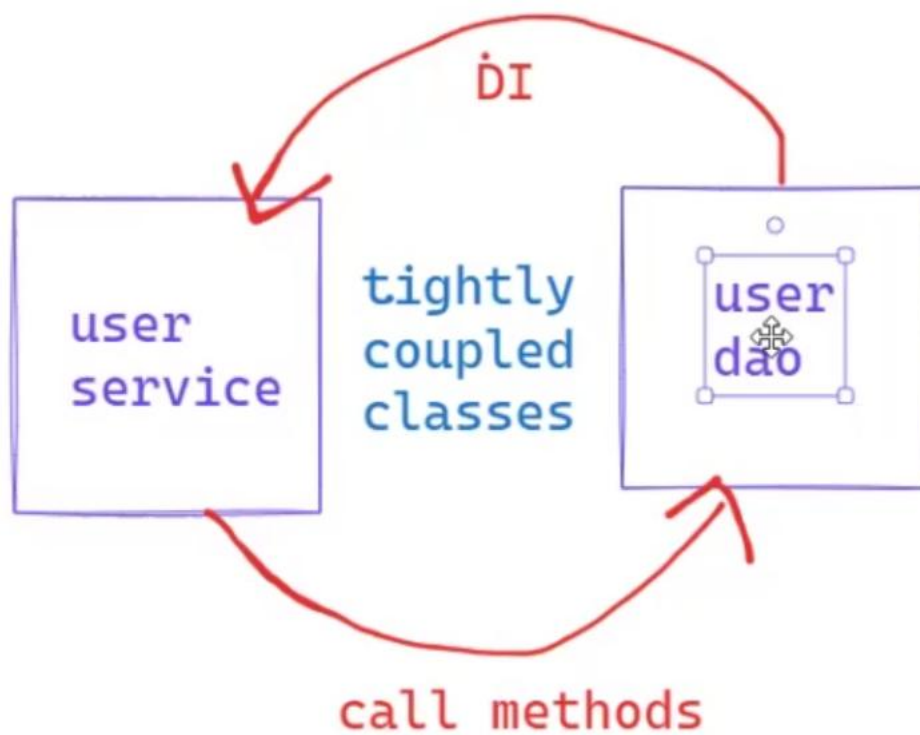
```
User.java x Test.java
1 package in.ashokit;
2
3 public class User {
4
5     private int age;
6
7     public void printAge() {
8         System.out.println("Age :: "+ age);
9     }
10
11 }
12 |
```

```

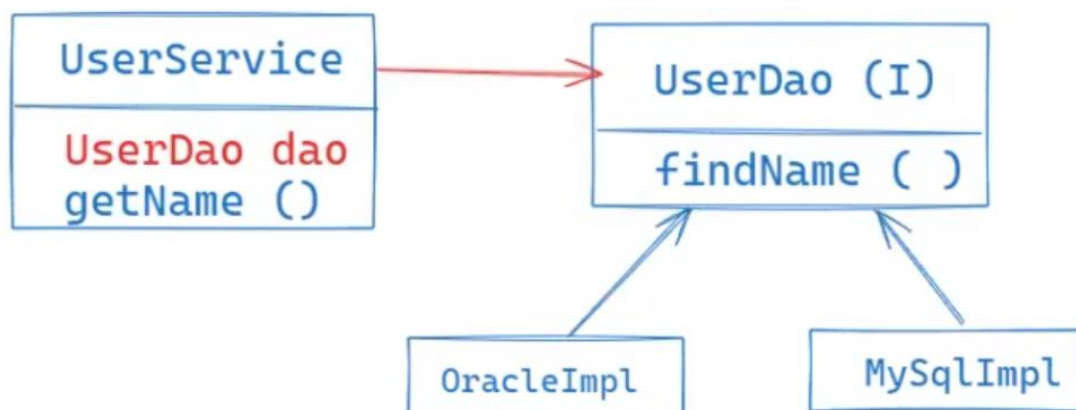
8      public static void main(String[] args) throws Exception {
9
10         Class<?> clz = Class.forName("in.ashokit.User");
11
12         Object obj = clz.newInstance();
13         User user = (User) obj;
14
15         Field field = clz.getDeclaredField("age");
16         field.setAccessible(true);
17
18         field.set(user, 25); // setting value to variable
19
20         user.printAge();
21
22     }

```





Best Practice – Loosely Coupled classes



Date : 06-Aug-2024 Topic : Runners

Runners in Spring boot

❖ Runners are used to execute any logic only once when the boot application got started.

❖ Runners will be called by SpringApplication.run () method.

❖ use cases:

1) delete temporary tables data when app started

2) load static tables data into cache memory when app started.

❖ We have 2 types of runners in springboot

1) ApplicationRunner (FI)

- run (..)

2) CommandLineRunner (FI)

- run (..)

@Component

public class MyAppRunner implements ApplicationRunner{

 @Override

 public void run(ApplicationArguments args) throws Exception {

 System.out.println("MyAppRunner :: run () method called..");

```

    }
}
-----

```

@Component

public class MyCmdRunner implements CommandLineRunner{

@Override

public void run(String... args) throws Exception {

System.out.println("MyCmdRunner :: run() method called..");

}

}

Bean life cycle

Q) What is spring bean ?

- ❖ The java class which is managed by ioc container is called as spring bean.
- ❖ IOC container will take care of bean life cycle
 - creating object
 - managing object
 - destroying object

Note: We have several annotations to represent java class as spring bean.

@Component
@Service
@Repository
@Bean
@Controller
@RestController

- ❖ When iOC container managing bean life cycle we can execute life cycle methods using below annotations

- a) @PostConstruct (after obj creation)
- b) @PreDestroy (before obj deletion)

@Component

```
public class ReportService {
```

```
    public ReportService() {  
        System.out.println("ReportService :: Constructor");  
    }  
}
```

@PostConstruct

```
public void init() {  
    System.out.println("ReportService - init () called..");  
}
```

@PreDestroy


```
public void destroy() {  
    System.out.println("ReportService - destroy () called..");  
}  
  
}
```

Stereotype annotation

@Component -> generic classes

@Service -> business logic classes

@Repository -> dao classes

SI vs CI vs FI

❖ CI is better

Summary

1) What is Spring boot

2) Advantages with Spring Boot

- pom starters
- auto configuration
- embedded servers
- actuators
- component scanning

- 3) Spring Boot application development
- 4) Spring Boot application project folder structure
- 5) Start class in spring boot
- 6) Internals of SpringApplication.run () method
- 7) What is @SpringBootApplication annotation
- 8) What is component scanning
 - base package
 - sub package
- 9) How to represent java class as Spring Bean
 - @Component
 - @Configuration
 - @Bean
- 10) Bean Scopes
 - singleton
 - prototype
 - request
 - session
- 11) Autowiring
 - byName
 - byType
 - constructor
- 12) Bean Life cycle
 - @PostConstruct
 - @PreDestroy
- 13) Runners in Spring boot
- 14) Banner in Spring Boot

15) IOC & DI

- setter injection
- constructor injection
- field injection

16) Spring boot annotation

@Component
@Service
@Repository
@Configuration
@Bean
@Scope
@Autowired
@Qualifier
@Primary
@PostConstruct
@PreDestroy

Assignment

1) Database Setup

- MySQL DB Server
- MySQL workbench (Client s/w)

Ref : <https://www.youtube.com/watch?v=EsAIXPlsyQg>

2) Developing First Data JPA app to insert emp record into mysql database table.

Ref : <https://www.youtube.com/watch?v=ZGKHCJsp4hg>